

벨만 방정식과 최적성

무한한 미래를 유한한 연립방정식으로 바꾸는 재귀 구조, 그리고 $\max$ 가 들어올 때 달라지는 것

sajacaros
sajacall82@naver.com

2026년 8월 25일

초록

앞 장에서 마르코프 결정 과정의 목표를 상태 가치 함수 $v_\pi(s) = \mathbb{E}_\pi[G_t | S_t = s]$ 로 정의하였으나, 이를 정의대로 계산하려면 무한히 이어지는 궤적을 모두 펼쳐야 하고 최적 정책을 찾으려면 $|S|^{|S|}$ 개의 정책을 모두 평가해야 했다. 벨만 방정식(Bellman equation)은 이 두 벽을 동시에 허문다. 수익의 재귀 구조 $G_t = R_t + \gamma G_{t+1}$ 한 줄에서 출발하여, 무한한 미래를 “한 걸음의 보상 + 이웃 상태의 가치”로 접어 넣으면 가치 함수는 미지수 $|S|$ 개짜리 연립일차방정식의 해가 된다. 본고는 이 도출을 백업 다이어그램의 확률 셈법에서부터 따라간 뒤, 행동 가치 함수 $q_\pi(s, a)$ 를 경유하여 $\sum_a \pi(a | s)$ 를 $\max_a$ 로 바꾼 벨만 최적 방정식에 이르는 경로를 정리한다. 원서[1] 3장은 수식 중심이어서 실행 가능한 코드를 제공하지 않으므로, 두 칸짜리 그리드 월드와 3×4 그리드 월드를 소재로 예제 네 편을 구성하였다. 본고의 절 구성은 원서 3장의 목차를 그대로 따르며, 원서에 없는 논의는 별도의 절로 떼지 않고 그것이 필요해지는 자리의 본문에 “[보충]”으로 표시해 두었다. 그렇게 남긴 것은 셋이며, 그 가운데 둘이 다음 장으로 곧장 이어진다. 첫째, 무작위 정책의 Q 를 한 번 탐욕화하면 3×4 그리드의 10개 상태 전부에서 가치가 오르지만 (평균 $-0.212 \rightarrow 0.760$) 세 상태는 여전히 최적(0.826)에 못 미쳐, “한 번의 탐욕화”와 “반복”을 가르는 간극이 수치로 드러난다. 둘째, 벨만 최적 방정식의 손풀이가 전제하 네 가지 결정적 정책 가정 중 자기일관인 것이 하나뿐임을 전수 확인하여, $\max$ 를 임의로 지운 식의 해는 최적 가치가 아니라 그 나쁜 정책의 가치일 뿐임을 보인다.

주제어: 강화 학습, 벨만 방정식, 벨만 최적 방정식, 행동 가치 함수, 연립일차방정식, 반복 대입, 고정점, 탐욕 정책

차례

서론	2
3.1 벨만 방정식 도출	3
3.1.1 확률과 기댓값(사전 준비)	3
3.1.2 벨만 방정식 도출	4
3.2 벨만 방정식의 예	6
3.2.1 두 칸짜리 그리드 월드	6
3.2.2 벨만 방정식의 의미	8
3.3 행동 가치 함수(Q 함수)와 벨만 방정식	9
3.3.1 행동 가치 함수	9
3.3.2 행동 가치 함수를 이용한 벨만 방정식	10
3.4 벨만 최적 방정식	12
3.4.1 상태 가치 함수의 벨만 최적 방정식	12
3.4.2 행동 가치 함수의 벨만 최적 방정식	13

3.5 벨만 최적 방정식의 예	13
3.5.1 벨만 최적 방정식 적용	13
3.5.2 최적 정책 구하기	15
3.6 정리	16
재현 정보	17

서론

전수 탐색의 벽

앞 장[2]에서 마르코프 결정 과정(MDP)을 정식화하고 목표를 상태 가치 함수

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s], \quad G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots \quad (1)$$

로 정의하였다. 그리고 두 칸짜리 그리드 월드에서 최적 정책을 찾을 때는 결정적 정책 네 가지를 모두 나열하여 각각의 가치를 500걸음 시뮬레이션으로 재는 전수 탐색을 썼다.

이 방법은 두 군데에서 막힌다. 하나는 정책 하나의 가치를 재는 일이 끝나지 않는다는 것이다. 식 (1)의 G_t 는 무한급수이므로 정의대로는 끝까지 더할 수 없고, 500걸음에서 자르는 것은 어디까지나 근사다. 다른 하나는 재야 할 정책의 수다. 결정적 정책의 수는 $|A|^{|S|}$ 이므로 상태가 100개이고 행동이 4가지면 $4^{100} \approx 1.6 \times 10^{60}$ 가지를 세어야 한다. 두 칸 세계에서 네 가지를 세어 본 것은 문제가 작았기 때문이지 방법이 좋았기 때문이 아니다.

그림 1이 첫 번째 벽을 그대로 보여준다. 앞 장에서 그린 백업 다이어그램인데, 맨 아래에서 가지가 점선으로 계속 이어진다. 수익을 정의대로 구하려면 이 점선을 끝까지 따라가야 한다.

그림 3-1 결정적 백업 다이어그램(왼쪽)과 확률적 백업 다이어그램(오른쪽)

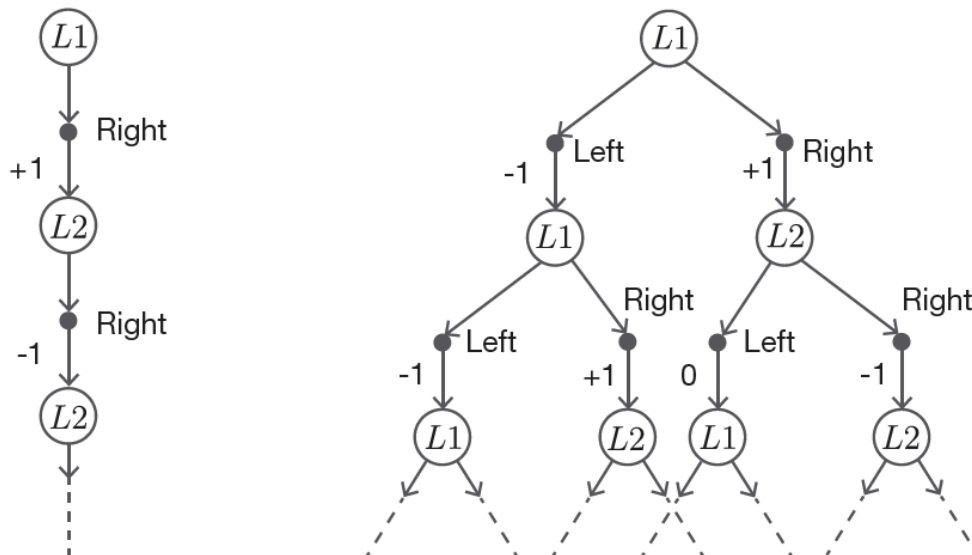


그림 1. 결정적 정책(왼쪽)과 확률적 정책(오른쪽)의 백업 다이어그램. 맨 아래에서 가지가 점선으로 계속 이어지는 것이 문제다. 벨만 방정식의 발상은 한 단계만 내려가고 그 아래 전체를 가치 함수 하나로 대신하는 것이다. 출처: 원서 그림 3-1.

벨만 방정식의 착상은 단순하다. 점선 아래에 있는 것도 결국 “그 상태에서 시작하는 수익의 기댓값”이므로 $v_{\pi}(s')$ 자체다. 그렇다면 무한한 가지를 펼치는 대신 한 층만 내려간 뒤 나머지를

$v_\pi(s')$ 로 바꿔 쓰면 된다. 그 대가로 v_π 가 등식의 양변에 나타나지만, 그것은 무한급수가 아니라 유한한 연립방정식이다.

본고의 구성과 기여

본고는 원서 3장의 목차를 그대로 따른다. §3.1에서 백업 다이어그램의 확률 셈법을 정리한 뒤 수익의 재귀 구조로부터 벨만 방정식을 도출하고, §3.2에서 그 식을 두 칸 세계와 3×4 그리드에 적용하여 연립일차방정식으로 푼다. §3.3에서 행동 가치 함수를 도입하여 V 와 Q 의 상호 변환을 정리하고, §3.4에서 $\sum_a \pi(a | s)$ 를 $\max_a$ 로 바꾼 벨만 최적 방정식을 세운 뒤 §3.5에서 그것을 다시 두 칸 세계에 적용한다. §3.6이 정리다.

원서 3장에는 소스 코드가 없으므로 예제 네 편을 새로 구성하였다(표 11). 원서에 없는 논의는 절을 따로 만들어 밀어 두지 않고, 그것이 필요해지는 자리의 본문에 “[보충]”으로 표시해 두었다. 그렇게 남긴 것은 셋이다. §3.2.2 끝에서는 정확해를 주는 직접 해법을 두고도 다음 장이 왜 반복법으로 가는지 연산량과 기억 용량으로 건준다. §3.3.2 끝에서는 3×4 그리드의 Q 를 한번 탐욕화해 보는데, 4장 정책 반복법이 왜 “한 번”이 아니라 “반복”인지가 거기서 나온다. §3.5.1 끝에서는 손풀이가 전제한 네 가지 가정을 전부 풀어, 벨만 최적 방정식을 푼다는 일의 논리 구조를 드러낸다.

반면 축소 사상의 형식적 증명, 수렴률의 고윳값 분해, 손익분기 상태 수의 닫힌 식 같은 논의는 이 장의 줄거리를 따라가는 데 꼭 필요하지 않아 덜어냈다. 반복 대입이 왜 통하는지는 §3.2.2에서 “오차가 한 걸음마다 γ 배 이하로 줄어든다”는 사실과 고정점이라는 말로만 짚고 넘어간다.

3.1 벨만 방정식 도출

3.1.1 확률과 기댓값(사전 준비)

벨만 방정식은 결국 기댓값 계산이므로, 먼저 백업 다이어그램에서 기댓값을 읽는 연습을 해 둔다. 가장 단순한 예는 주사위 한 번 던지기다.

그림 3-2 주사위의 백업 다이어그램

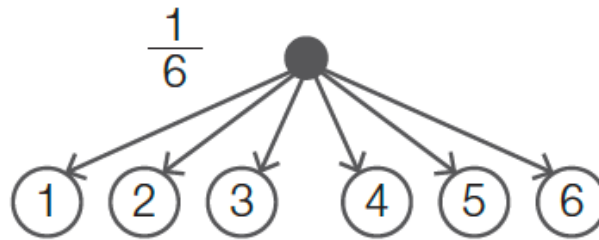


그림 2. 주사위 한 번 던지기의 백업 다이어그램. 검은 점에서 여섯 갈래로 뻗고 가지마다 확률 $1/6$ 이 붙는다. 출처: 원서 그림 3-2.

가지에 붙은 확률과 끝의 값을 곱해 모두 더한 것이 기댓값이다.

$$\mathbb{E}[X] = \sum_x x p(x) = \frac{1 + 2 + 3 + 4 + 5 + 6}{6} = 3.5. \quad (2)$$

다음은 두 단계짜리 문제다. 주사위를 던져 홀수가 나오면 공정한 동전을, 짝수가 나오면 앞면이 잘 나오는 동전을 던진다. 동전이 앞면이면 주사위 눈만큼 보상을 받고 뒷면이면 0이다.

그림 3-3 주사위와 동전을 순서대로 던지는 문제

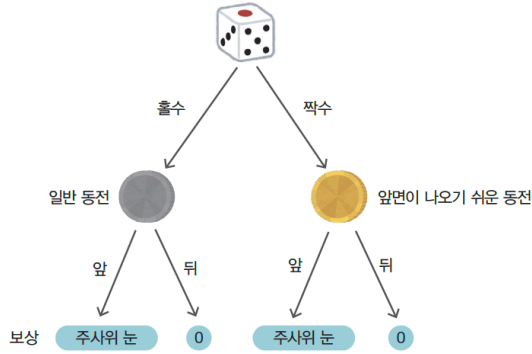


그림 3-4 주사위와 동전을 순서대로 던지는 문제의 백업 다이어그램

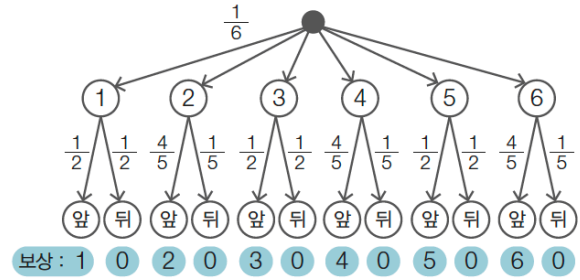


그림 3. 두 단계짜리 문제(왼쪽)와 이를 하나의 백업 다이어그램으로 편 것(오른쪽). 두 번째 단계의 확률이 첫 번째 단계의 결과에 의존한다는 점이 MDP와 같은 구조다. 홀수 가지의 앞면 확률은 $1/2$, 짝수 가지는 $4/5$ 이다. 출처: 원서 그림 3-3, 3-4.

이때 기댓값은 맨 위에서 맨 아래까지 가지의 확률을 모두 곱하고 거기에 마지막 값을 곱해 더한 것이다.

$$\begin{aligned} \mathbb{E}[R] &= \frac{1}{6} \left(1 \cdot \frac{1}{2} + 2 \cdot \frac{4}{5} + 3 \cdot \frac{1}{2} + 4 \cdot \frac{4}{5} + 5 \cdot \frac{1}{2} + 6 \cdot \frac{4}{5} \right) \\ &= \frac{1}{6} (0.5 + 1.6 + 1.5 + 3.2 + 2.5 + 4.8) = \frac{14.1}{6} = 2.35. \end{aligned} \quad (3)$$

뒷면 가지는 보상이 0이라 합에서 사라지므로 12개 가지 중 6개 항만 남았다.

식 (3)에서 확인할 것은 두 가지다. 첫째, 가지를 따라 확률을 곱해 더한다는 셈법이 층이 늘어도 그대로 통한다. 둘째, 두 번째 층의 확률이 첫 번째 층의 결과에 따라 달라진다. 이 두 성질이 그대로 벨만 방정식의 유도에 쓰인다. MDP에서는 첫 번째 층이 정책이고 두 번째 층이 상태 전이일 뿐이다.

3.1.2 벨만 방정식 도출

수익의 재귀 구조

출발점은 수익의 정의다.

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 R_{t+3} + \dots \quad (4)$$

한 시각 뒤의 수익도 같은 모양이다.

$$G_{t+1} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \quad (5)$$

식 (4)에서 첫 항을 떼고 나머지를 γ 로 묶으면 식 (5)이 그대로 나타난다.

$$G_t = R_t + \gamma G_{t+1}. \quad (6)$$

이 한 줄에서 이 장의 모든 식이 나온다. 식 (6)이 하는 일은 무한히 이어지는 합을 한 걸음과 나머지 전부로 나눈 것뿐이지만, 나머지 전부가 원래 문제와 같은 모양이라는 점이 결정적이다. 앞장에서 이 식은 보상 열을 뒤에서부터 접어 올리는 계산 요령으로만 쓰였는데, 이제 그것이 방정식이 된다.

이제 식 (6)을 가치 함수의 정의

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s] \quad (7)$$

에 대입하고 기댓값의 선형성으로 두 항을 나눈다.

$$v_\pi(s) = \mathbb{E}_\pi[R_t \mid S_t = s] + \gamma \mathbb{E}_\pi[G_{t+1} \mid S_t = s]. \quad (8)$$

그림 3-6 전개 중인 식

$$v_{\pi}(s) = \mathbb{E}_{\pi}[R_t | S_t = s] + \gamma \mathbb{E}_{\pi}[G_{t+1} | S_t = s]$$

첫 번째 항 전개
다음은 이쪽 항

$$\sum_{a, s'} \pi(a|s) p(s'|s, a) r(s, a, s')$$

그림 4. 식 (8)의 두 항. 하늘색으로 칠해진 두 덩어리를 각각 전개한다. 왼쪽은 한 걸음의 기댓값이므로 §3.1.1의 샘플법을 그대로 적용하면 되고, 오른쪽은 그 뒤에 처리한다. 출처: 원서 그림 3-6.

두 층의 확률: 정책과 상태 전이

식 (8)의 두 항을 전개하려면 어떤 확률이 걸려 있는지 알아야 한다. 그림 5이 그것을 보여준다.

그림 3-5 상태와 행동의 관계

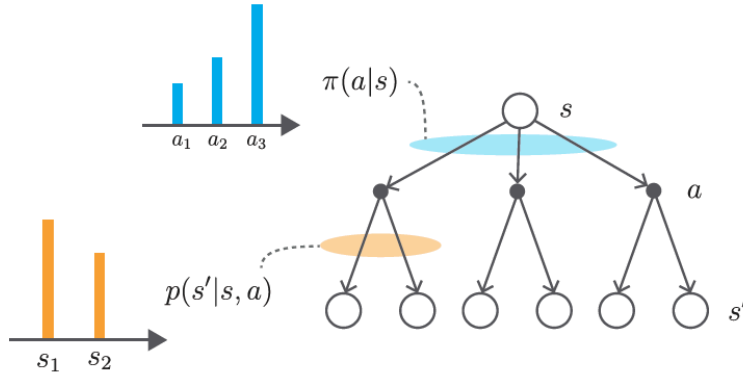


그림 5. 한 걸음에 걸린 두 개의 확률분포. 하늘색(상태 → 행동)은 정책 $\pi(a | s)$ 이고, 주황색(행동 → 다음 상태)은 상태 전이 확률 $p(s' | s, a)$ 다. 왼쪽의 막대그래프 두 개는 각각이 확률분포임을 강조한 것이다. 출처: 원서 그림 3-5.

§3.1.1의 두 단계 문제와 구조가 같다. 맨 위에서 s' 까지 내려가는 한 가지의 확률은 두 층의 확률을 곱한 $\pi(a | s) p(s' | s, a)$ 이고, 그 끝에 붙은 값이 보상 $r(s, a, s')$ 이다. 따라서 식 (8)의 첫 항은

$$\mathbb{E}_{\pi}[R_t | S_t = s] = \sum_a \sum_{s'} \pi(a | s) p(s' | s, a) r(s, a, s') \quad (9)$$

이 된다. 둘째 항도 같다. 상태가 s' 으로 바뀌고 나면 그 뒤의 기대 수익은 정의상 $v_{\pi}(s')$ 이므로

$$\mathbb{E}_{\pi}[G_{t+1} | S_t = s] = \sum_a \sum_{s'} \pi(a | s) p(s' | s, a) v_{\pi}(s') \quad (10)$$

이다. 여기가 유도의 핵심이다. G_{t+1} 은 그 자체로 무한급수인데, 그 무한급수의 기댓값을 다시 펼치지 않고 $v_{\pi}(s')$ 이라는 이름으로 대신했다. 그림 1의 점선이 여기서 잘린다.

벨만 방정식

식 (9)과 (10)을 식 (8)에 넣고 공통의 $\sum$ 으로 묶으면 벨만 방정식을 얻는다.

$$v_{\pi}(s) = \sum_a \sum_{s'} \pi(a | s) p(s' | s, a) \{ r(s, a, s') + \gamma v_{\pi}(s') \}. \quad (11)$$

이중 합을 안쪽으로 분리해 쓰면 코드로 옮기기 좋은 형태가 된다.

$$v_{\pi}(s) = \sum_a \pi(a | s) \sum_{s'} p(s' | s, a) \{ r(s, a, s') + \gamma v_{\pi}(s') \}. \quad (12)$$

바깥쪽 $\sum_a$ 는 정책에 대한 평균이고 안쪽 $\sum_{s'}$ 은 상태 전이에 대한 평균이다.

상태 전이가 결정적이어서 $s' = f(s, a)$ 로 정해지면 안쪽 합이 사라진다.

$$v_{\pi}(s) = \sum_a \pi(a | s) \{ r(s, a, f(s, a)) + \gamma v_{\pi}(f(s, a)) \}. \quad (13)$$

본고가 다루는 두 환경은 모두 상태 전이가 결정적이므로 실제 계산에는 식 (13)을 쓴다.

식 (11)이 말하는 바는 한 문장으로 줄어든다.

어떤 상태의 가치는 (한 걸음 보상의 기댓값) + $\gamma \times$ (이웃 상태 가치의 기댓값)이다.

무한한 미래를 다 볼 필요가 없어졌다는 것이 얻은 것이고, v_{π} 가 좌변과 우변에 동시에 나타나는 자기참조 구조가 치른 대가다. 다음 절에서 보듯 그 대가는 연립방정식을 푸는 정도의 값이다.

3.2 벨만 방정식의 예

3.2.1 두 칸짜리 그리드 월드

앞 장에서 쓴 두 칸짜리 그리드 월드를 그대로 가져온다. 이번엔 평가할 정책은 L1과 L2 어디에서나 Left와 Right를 0.5씩 고르는 무작위 정책이다.

그림 3-8 넓게 퍼져나가는 백업 다이어그램(이번 문제에서 상태는 결정적으로 전이됨)

그림 3-7 두 칸짜리 그리드 월드(벽에 부딪히면 -1, 사과를 얻으면 +1, 사과를 계속 생성)

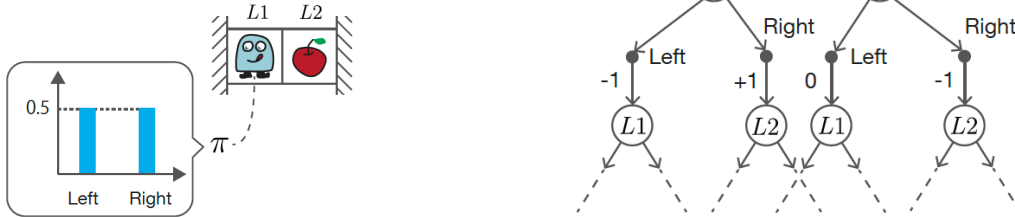


그림 6. 두 칸짜리 그리드 월드와 무작위 정책(왼쪽), 그 백업 다이어그램(오른쪽). 왼쪽 막대그래프 두 개의 높이가 같은 것이 $\pi(a | s) = 0.5$ 이다. 오른쪽에서 상태에서 나오는 가지는 둘로 갈라지지만 행동에서 나오는 화살표는 하나뿐인데, 이것이 상태 전이가 결정적이라는 뜻이며 따라서 식 (13)을 쓸 수 있다. 출처: 원서 그림 3-7, 3-8.

환경의 규칙은 앞 장과 같다. 벽에 부딪히면 -1, L2로 이동하면 사과를 먹어 +1, L2에서 L1로 가면 0이며 $\gamma = 0.9$ 다. 이제 상태마다 벨만 방정식을 한 줄씩 세운다.

그림 7을 그대로 읽으면 다음 두 식이 나온다.

$$v_{\pi}(L1) = 0.5 \{ -1 + 0.9 v_{\pi}(L1) \} + 0.5 \{ 1 + 0.9 v_{\pi}(L2) \}, \quad (14)$$

$$v_{\pi}(L2) = 0.5 \{ 0 + 0.9 v_{\pi}(L1) \} + 0.5 \{ -1 + 0.9 v_{\pi}(L2) \}. \quad (15)$$

그림 3-9 $v_\pi(L1)$ 을 구하기 위한 백업 다이어그램

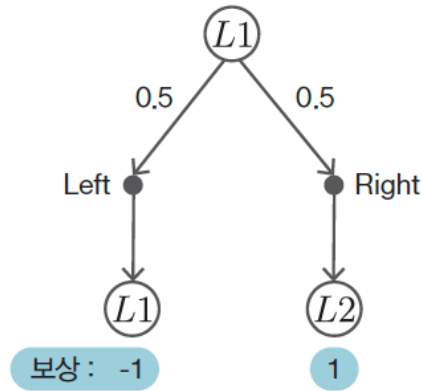


그림 3-10 $v_\pi(L2)$ 을 구하기 위한 백업 다이어그램

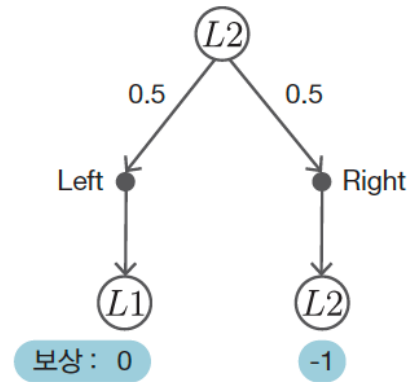


그림 7. $v_\pi(L1)$ (왼쪽)과 $v_\pi(L2)$ (오른쪽)에 대한 한 단계짜리 다이어그램. 그림 6 오른쪽의 맨 윗부분만 떼어낸 것으로, 아래가 점선으로 이어지지 않고 L1, L2에서 끊겨 있다. 그 아래 전체를 $v_\pi(L1)$, $v_\pi(L2)$ 로 대신하는 것이 벨만 방정식의 요령이다. 출처: 원서 그림 3-9, 3-10.

정리하면 미지수 두 개짜리 연립일차방정식이다.

$$\begin{cases} -0.55 v_\pi(L1) + 0.45 v_\pi(L2) = 0, \\ 0.45 v_\pi(L1) - 0.55 v_\pi(L2) = 0.5. \end{cases} \quad (16)$$

풀면 $v_\pi(L1) = -2.25$, $v_\pi(L2) = -2.75$ 다. 무작위로 걸으면 벽에 부딪히는 일이 잦아 값이 음수가 된다.

식 (16)을 세운 과정은 정책이 바뀌어도 달라지지 않는다. 앞 장이 전수 탐색으로 찾아낸 결정적 정책 $\mu = (\text{Right}, \text{Left})$ 에 대해 같은 자리에 다른 확률과 보상을 넣어 두 식을 세우고 풀면 표 1의 아래 행이 나온다. 예제 s01_bellman_linear.py가 두 정책의 연립방정식을 모두 풀어 확인한다.

표 1. 두 칸짜리 그리드 월드의 상태 가치($\gamma = 0.9$). 무작위 정책의 값은 식 (16)의 해와 같고, 결정적 정책 μ 의 값은 앞 장이 500걸음 시뮬레이션으로 얻었던 값과 같다.

정책	$v(L1)$	$v(L2)$	비고
무작위 $\pi(a s) = 0.5$	-2.2500	-2.7500	원서 3.2절의 값
$\mu = (\text{Right}, \text{Left})$	5.2632	4.7368	앞 장의 전수 탐색 최적 정책

표 1의 아래 행이 이 장의 요점이다. 앞 장에서 5.2632를 얻으려면 정책을 고정하고 500걸음을 시뮬레이션해야 했다. 여기서는 2×2 연립방정식을 한 번 풀었다. 근사가 정확해로 바뀌었고 걸음 수라는 임의의 절단 지점도 사라졌다.

3 × 4 그리드 월드로

상태가 12개가 되면 손으로 풀 수 있는 크기를 넘어서지만, 세우는 방법은 조금도 달라지지 않는다. 상태마다 식을 한 줄씩 쓰면 미지수 12개짜리 연립일차방정식이고, 같은 예제의 후반부가 그것을 푼다. 이 환경은 다음 장에서 반복 계산으로 다시 풀 문제다.

표 2에서 오른쪽 아래 -0.7857이 가장 낮다. 폭탄(1, 3) 바로 아래이면서 목표에서 멀어, 무작위로 걸으면 폭탄에 빨려 들어갈 확률이 높은 자리다. 목표 상태 왼쪽의 0.2055가 가장 높은 것도 같은 이유의 뒤집힌 표현이다.

이 값들은 다음 장의 반복 정책 평가가 구하는 값과 최대 오차 4.644×10^{-10} 로 일치한다. 반복 계산에 쓴 종료 문턱이 10^{-10} 이었으므로 이는 두 방법이 같은 방정식을 서로 다른 방식으로 푼 것임을 보여준다.

표 2. 3×4 그리드 월드에서 무작위 정책($\pi(a | s) = 0.25$)의 상태 가치($\gamma = 0.9$). (0, 3)은 목표 상태(가치 0), (1, 1)은 벽, (1, 3)은 폭탄이다. 12원 연립방정식의 정확해다.

	$w = 0$	$w = 1$	$w = 2$	$w = 3$
$h = 0$	0.0257	0.0946	0.2055	0.0000
$h = 1$	-0.0318	(벽)	-0.4980	-0.3727
$h = 2$	-0.1034	-0.2210	-0.4368	-0.7857

3.2.2 벨만 방정식의 의의

벨만 방정식이 없다면 가치를 구하는 길은 두 가지뿐이고 둘 다 막혀 있다.

표 3. 벨만 방정식이 없을 때 $v_\pi(s)$ 에 이르는 두 경로. 어느 쪽도 끝나지 않는다.

방법	막히는 곳
수익의 정의대로 계산	무한히 이어지는 항을 모두 더해야 한다(앞 장은 500걸음에서 잘랐다)
모든 궤적을 나열해 기댓값	그림 6 오른쪽처럼 가치가 2^n 으로 폭발한다

벨만 방정식은 이 문제를 미지수 $|S|$ 개짜리 연립일차방정식으로 바꾼다. 상태가 100개면 100원 연립방정식이며, 유한하고 컴퓨터가 곧바로 풀 수 있는 크기다. 앞 장의 전수 탐색이 마주한 4^{100} 과는 차원이 다르다. 무한급수와 지수적 나열이라는 두 벽이 동시에 사라지는 자리가 여기도.

다만 정확해를 주는 이 길에도 한계가 있다. 미지수가 $|S|$ 개인 연립방정식을 정면으로 소거해 푸는 데는 계수를 담을 $|S| \times |S|$ 크기의 표가 필요하고 연산량이 $O(|S|^3)$ 이므로, 상태가 아주 많아지면 감당하기 어려워진다. 그래서 실제로는 우변에 계속 대입해 근사해로 다가가는 반복 대입을 쓰며, 그것이 다음 장의 동적 프로그래밍이다.

반복 대입이 왜 되는지는 벨만 방정식(식 (11))을 다시 보면 알 수 있다. 이 식은 좌변에도 v_π 가 있고 우변에도 v_π 가 있다. $x = f(x)$ 처럼 넣은 것이 그대로 다시 나오는 꼴의 방정식을 고정점 방정식이라 하고, 그것을 만족하는 x 를 f 의 고정점(fixed point)이라 한다. 익숙한 예가 $x = \cos x$ 다. 계산기에 아무 수나 넣고 $\cos$ 을 계속 누르면 0.739085...로 빨려 들어가는데, 그 값이 $\cos$ 의 고정점이다.

벨만 방정식을 푸는 일도 이와 같다. 다루는 것이 숫자 하나가 아니라 상태마다 값이 하나씩 적힌 표, 곧 길이 $|S|$ 짜리 벡터라는 점만 다르다. 지금 손에 든 표 v_k 를 잠정적인 가치 추정치로 믿고 식 (11)의 우변에 넣어 보자. 상태 s 에서 한 걸음만 실제로 굴러 보상 $r(s, a, s')$ 을 받은 다음, 그 이후는 도착한 칸의 값 $v_k(s')$ 을 그대로 갖다 쓰는 계산이다. 요컨대 한 걸음은 진짜로 계산하고 나머지는 지금의 추정치로 때우는 것이며, 그 결과를 모든 상태에 대해 적어 놓은 새 표가 v_{k+1} 이다.

$$v_{k+1}(s) = \sum_a \pi(a | s) \sum_{s'} p(s' | s, a) \{r(s, a, s') + \gamma v_k(s')\} \quad (17)$$

참값 v_π 를 넣었다면 때운 부분까지 참값이므로 나온 표도 v_π 다. 즉 v_π 는 이 되풀이의 고정점이고, $\cos$ 의 예가 그랬듯이 아무 표에서 출발해 계속 대입하는 것만으로 그 표에 이를 수 있다.

다만 아무 f 에서나 되는 것은 아니고 f 가 두 점 사이의 거리를 줄이는 함수여야 하는데($\cos$ 이 그렇다), 식 (17)의 우변이 바로 그런 함수다. 서로 다른 표를 두 개 넣어 보면 알 수 있다. v_k 가 들어가는 자리는 $\gamma v_k(s')$ 하나뿐이고, 그 앞에 붙은 $\sum_a \pi(a | s) \sum_{s'} p(s' | s, a)$ 는 합이 1인 확률로 평균을 내는 일이다. 두 표의 차이를 아무리 섞어 평균내도 그중 가장 큰 차이보다 커질 수 없으므로, 남는 것은 앞에 붙은 γ 다. 즉 한 번 대입할 때마다 두 표의 거리는 γ 배 이하로 줄어들며, 그 두 표 중 하나를 참값 v_π 로 두면 오차에 대한 다음 식이 된다.

$$\|v_k - v_\pi\|_\infty \leq \gamma^k \|v_0 - v_\pi\|_\infty \quad (18)$$

이 반복을 실제로 돌려 보는 예제가 `s02_bellman_iterative.py`이고 그림 8이 그 오차다. 로그 눈금에서 직선이라는 것은 오차가 지수적으로 줄어든다는 뜻이고, 그 기울기를 정하는 것이

γ 다. γ 가 1에 가까울수록 선이 완만해져 반복이 오래 걸린다는 사실이 두 그림의 차이에 그대로 나타난다.

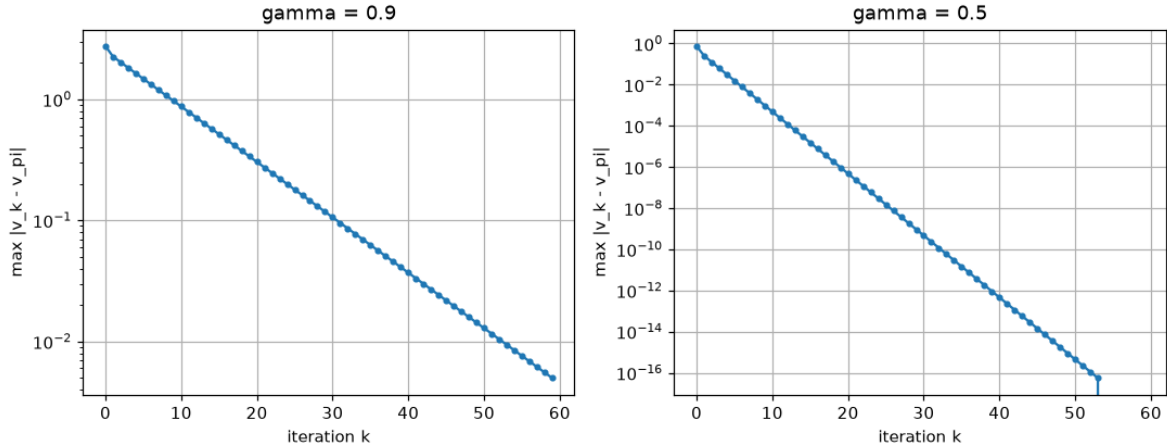


그림 8. 반복 대입의 오차(로그 눈금). 왼쪽($\gamma = 0.9$)은 60회를 돌려도 오차가 4.5×10^{-3} 남지만, 오른쪽($\gamma = 0.5$)은 53회 부근에서 선이 끊긴다. 그 지점이 0.5^k 가 배정밀도로 표현할 수 있는 크기 아래로 내려가 오차가 정확히 0이 된 곳이다. `s02_bellman_iterative.py`에서 생성.

[보충] 그렇다면 언제 반복법으로 갈아타는가

직접 해법과 반복법은 주는 것이 서로 다르다. 직접 해법은 정확해를 유한 단계에 주는 대신 $|S|$ 원 연립방정식을 소거하는 데 $|S|$ 의 세제곱에 비례하는 연산이 들고, 계수를 담은 $|S| \times |S|$ 크기의 표를 통째로 들고 있어야 한다. 반복법은 근사해만 주는 대신 한 번의 갱신 비용이 상태 수에 선형이고, 기억할 것은 길이 $|S|$ 의 벡터 두 개뿐이며, 필요한 정확도만큼만 돌면 된다.

예제 `s01_bellman_linear.py`가 직접 해법을 쓴 것은 그래서 옳다. 상태가 2개와 12개뿐인 본고의 두 환경은 그 우위가 분명한 크기다. 그러나 이 우위는 세제곱으로 커지는 항에 기대고 있어 금세 뒤집힌다. 먼저 무너지는 쪽은 시간이 아니라 공간이다. $|S| = 10^4$ 이면 계수 표 하나가 배정밀도로 0.8GB이고 $|S| = 10^6$ 이면 8TB라 어떤 기계에도 올라가지 않는 반면, 반복법이 들고 있어야 하는 것은 여전히 벡터 두 개다. 다음 장의 동적 프로그래밍이 반복법을 택하는 이유가 여기에 있고, 5장 이후 상태 공간이 더 커질 때 가치 함수를 신경망으로 근사하게 되는 출발점도 같은 자리다.

3.3 행동 가치 함수(Q 함수)와 벨만 방정식

3.3.1 행동 가치 함수

지금까지의 $v_\pi(s)$ 는 “상태 s 에서 정책 π 를 따를 때”의 기대 수익이다. 여기서 첫 행동만 정책의 손에서 빼앗아 자유롭게 지정한 것이 행동 가치 함수(action-value function), 흔히 Q 함수다.

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]. \quad (19)$$

식 (7)과 비교하면 조건에 $A_t = a$ 가 하나 더 붙었을 뿐이다.

v_π 를 유도할 때와 똑같은 순서를 밟는다. 식 (6)을 대입하고 선형성으로 나눈 뒤 두 번째 항에 $v_\pi(s')$ 를 쓰면

$$q_\pi(s, a) = \sum_{s'} p(s' \mid s, a) \{ r(s, a, s') + \gamma v_\pi(s') \} \quad (20)$$

그림 3-11 상태 가치 함수와 Q 함수의 백업 다이어그램

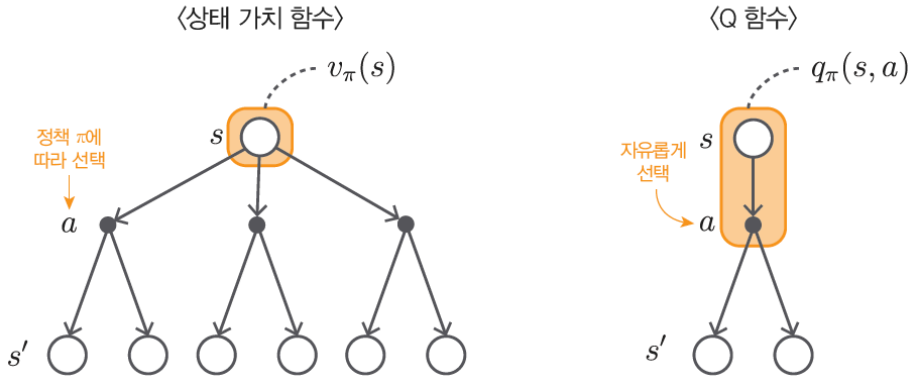


그림 9. v_π (왼쪽)와 q_π (오른쪽)의 차이. 주황색으로 감싼 범위가 왼쪽은 상태 s 만이고 오른쪽은 s 와 행동 a 까지다. 왼쪽에서는 첫 행동도 정책이 고르므로 갈래가 여럿이지만, 오른쪽에서는 첫 행동이 지정되어 갈래가 하나로 고정되고 그 아래부터 π 를 따른다. 이 “첫 행동만 자유”라는 차이가 4장 정책 개선과 6장 Q러닝의 근거가 된다. 출처: 원서 그림 3-11.

를 얻는다. 식 (12)에서 바깥쪽 $\sum_a \pi(a | s)$ 만 떼어낸 모양이다. 정책이 고르던 자리를 우리가 대신 골랐으니 그 평균이 사라지는 것은 당연하다. 반대 방향은 그 평균을 도로 씌우면 된다.

$$v_\pi(s) = \sum_a \pi(a | s) q_\pi(s, a). \quad (21)$$

식 (20)과 (21)은 V 와 Q 가 같은 정보의 두 표현임을 말한다. 그렇다면 왜 둘 다 필요한가. 표 4가 그 답이다.

표 4. V 와 Q 의 비교. 마지막 열이 5장 이후 모델 없는 기법이 모두 Q 를 쓰는 이유다.

	입력	첫 행동을 고르는 주체	행동을 고르려면
$v_\pi(s)$	상태	정책 π	p 와 r 를 알아야 함(식 (32))
$q_\pi(s, a)$	상태 + 행동	내가 지정	$\arg \max_a q_\pi(s, a)$ 면 끝
기억 용량	$ S $ 개	—	$ S \times A $ 개

Q 는 $|A|$ 배의 기억 용량을 쓰는 대신, 행동을 고를 때 환경 모델 p 와 r 를 참조하지 않아도 되게 해 준다. 환경 모델을 모르는 채로 경험만 가지고 배우는 5장 이후의 기법들이 예외 없이 Q 를 추정하는 것은 이 한 줄 때문이다.

3.3.2 행동 가치 함수를 이용한 벨만 방정식

식 (20)의 우변에는 아직 v_π 가 남아 있다. 여기에 식 (21)을 대입하면 Q 만으로 닫힌 식이 된다.

$$q_\pi(s, a) = \sum_{s'} p(s' | s, a) \left\{ r(s, a, s') + \gamma \sum_{a'} \pi(a' | s') q_\pi(s', a') \right\}. \quad (22)$$

식 (11)과 견주면 안쪽에 $\sum_{a'} \pi(a' | s')$ 가 새로 생겼다. 다음 상태에서 고를 행동까지 정책으로 평균낸다는 뜻이다. 이 부분은 6장 SARSA의 갱신식에 그대로 나타난다.

예제 s03_q_function.py가 두 칸 세계에서 이 관계들을 수치로 확인한다. 표 5가 그 결과다.

표 5에서 눈여겨볼 것은 $q_\pi(L1, Right) = -1.4750$ 이 $v_\pi(L1) = -2.2500$ 보다 크다는 점이다. L1에서 무작위로 고르는 것보다 Right를 고르는 편이 낫다는 뜻이며, 그 차이 0.775가 곧 개선의 여지다. L2에서도 Left(-2.0250)가 평균(-2.7500)보다 낫다. V 만 들여다볼 때는 보이지 않던 “어느 방향으로 고치면 되는가”가 Q 에서는 표를 훑는 것만으로 읽힌다.

표 5. 두 칸짜리 그리드 월드에서 무작위 정책의 Q 함수($\gamma = 0.9$). 마지막 열은 식 (21)으로 복원한 v_π 이며 표 1의 값과 일치한다.

s	$q_\pi(s, \text{Left})$	$q_\pi(s, \text{Right})$	$\frac{1}{2}(q_\pi(s, \text{L}) + q_\pi(s, \text{R})) = v_\pi(s)$
L1	-3.0250	-1.4750	-2.2500
L2	-2.0250	-3.4750	-2.7500

[보충] 한 번의 탐욕화가 주는 것과 주지 못하는 것

그렇다면 Q 를 한 번 탐욕화하면 얼마나 좋아지는가. 3×4 그리드에서 무작위 정책의 q_π (표 2의 v_π 로부터 식 (20)로 얻은 것)를 탐욕화한 정책 $\pi'(s) = \arg \max_a q_\pi(s, a)$ 을 만들고, 그 정책의 가치 $v_{\pi'}$ 를 다시 연립방정식으로 정확히 풀어 비교하였다. 표 6가 결과다.

표 6. 3×4 그리드에서 한 번의 탐욕화가 주는 개선($\gamma = 0.9$). v_π 는 무작위 정책, $v_{\pi'}$ 는 이를 한 번 탐욕화한 정책, v_* 는 최적 가치다. 벽과 목표 상태는 제외하였다.

상태 s	$v_\pi(s)$	$v_{\pi'}(s)$	$v_*(s)$	개선 $v_{\pi'} - v_\pi$	남은 차이 $v_* - v_{\pi'}$
(0, 0)	0.0257	0.8100	0.8100	+0.7843	0.0000
(0, 1)	0.0946	0.9000	0.9000	+0.8054	0.0000
(0, 2)	0.2055	1.0000	1.0000	+0.7945	0.0000
(1, 0)	-0.0318	0.7290	0.7290	+0.7608	0.0000
(1, 2)	-0.4980	0.9000	0.9000	+1.3980	0.0000
(1, 3)	-0.3727	1.0000	1.0000	+1.3727	0.0000
(2, 0)	-0.1034	0.6561	0.6561	+0.7595	0.0000
(2, 1)	-0.2210	0.5905	0.7290	+0.8115	0.1385
(2, 2)	-0.4368	0.5314	0.8100	+0.9683	0.2786
(2, 3)	-0.7857	0.4783	0.7290	+1.2640	0.2507
평균	-0.2124	0.7595	0.8263	+0.9719	0.0668

두 가지를 읽을 수 있다.

주는 것: 모든 상태에서의 개선. 개선 열에 음수가 하나도 없다. 10개 상태 전부에서 가치가 올랐고 평균은 -0.2124에서 0.7595로 바뀌었다. 이것은 우연이 아니라 정책 개선 정리의 내용이다. π' 가 q_π 에 대해 탐욕적이면 $q_\pi(s, \pi'(s)) = \max_a q_\pi(s, a) \geq \sum_a \pi(a | s) q_\pi(s, a) = v_\pi(s)$ 이고, 이를 반복 전개하면 모든 상태에서 $v_{\pi'}(s) \geq v_\pi(s)$ 가 따라온다. 최댓값은 평균보다 작을 수 없다는 사실 하나가 전부다. 무작위 정책이라는 형편없는 출발점에서 단 한 번의 탐욕화로 최적 평균 0.8263의 92%에 도달한 셈이다.

주지 못하는 것: 최적성. 그러나 마지막 열에 0이 아닌 값이 셋 남아 있다. 아래쪽 세 칸 (2, 1), (2, 2), (2, 3)은 최적값에 각각 0.139, 0.279, 0.251 못 미친다. 정책을 직접 비교하면 이유가 분명해진다.

표 7. 한 번 탐욕화한 정책 π' 와 최적 정책 μ_* . 아래쪽 세 칸에서만 갈린다.

	$\pi' = \arg \max_a q_\pi(s, a)$				$\mu_* = \arg \max_a q_*(s, a)$			
	$w = 0$	$w = 1$	$w = 2$	$w = 3$	$w = 0$	$w = 1$	$w = 2$	$w = 3$
$h = 0$	RIGHT	RIGHT	RIGHT	GOAL	RIGHT	RIGHT	RIGHT	GOAL
$h = 1$	UP	WALL	UP	UP	UP	WALL	UP	UP
$h = 2$	UP	LEFT	LEFT	LEFT	UP	RIGHT	UP	LEFT

π' 는 아래쪽 줄에서 모두 왼쪽으로 돌아간다. 무작위 정책 아래에서는 오른쪽 열이 폭탄(1,3)의 영향으로 크게 음수였고(표 2의 $-0.4980, -0.7857$), 그 값을 근거로 탐욕화했으니 오른쪽 열을 피하는 것이 당연하다. 그러나 그 음수는 무작위로 걸을 때 폭탄에 빨려 들어가기 때문에 생긴 값이다. 정책이 이미 개선되고 나면 (2,2)에서 위로 올라가 (1,2) $\rightarrow$ (0,2) $\rightarrow$ 목표로 가는 짧은 길이 안전해진다. π' 는 낮은 가치 함수를 근거로 판단했으므로 이 사실을 알 수 없다.

여기서 다음 장의 필요성이 나온다. 탐욕화한 정책 π' 의 가치를 다시 평가하고 그 결과로 다시 탐욕화하면, 방금 안전해진 길이 값에 반영되어 정책이 또 바뀐다. 이 “평가 $\rightarrow$ 탐욕화”의 반복이 4장 정책 반복법이며, 표 6의 마지막 열에 남은 0.0668이 그 반복이 아직 회수하지 못한 몫이다. 한 번의 탐욕화는 정책 반복법의 첫 걸음일 뿐이지 그 전부가 아니다.

3.4 벨만 최적 방정식

지금까지는 주어진 정책 π 를 평가했다. 이제 최적 정책의 가치를 정책을 거치지 않고 직접 구한다.

3.4.1 상태 가치 함수의 벨만 최적 방정식

최적 정책 π_* 도 정책이므로 그 가치 함수 v_* 는 벨만 방정식을 만족한다.

$$v_*(s) = \sum_a \pi_*(a | s) \sum_{s'} p(s' | s, a) \{r(s, a, s') + \gamma v_*(s')\}. \quad (23)$$

여기에 앞 장에서 확인한 성질, 즉 최적 정책 중에는 결정적 정책이 있다는 사실을 쓴다.

그림 3-12 세 가지 행동 중 어떤 행동을 선택할까?

$$v_*(s) = \sum_a \pi_*(a | s) \sum_{s'} p(s' | s, a) \{r(s, a, s') + \gamma v_*(s')\}$$

$= \begin{cases} -2 & (a_1) \\ 0 & (a_2) \\ 4 & (a_3) \end{cases}$

그림 10. 안쪽 합(하늘색)의 값이 행동마다 $-2, 0, 4$ 로 계산된 상황. 결정적 정책이라면 π_* 는 가장 큰 a_3 에 확률 1을 줄 수밖에 없고, 그러면 $\sum_a \pi_*(a | s)(\dots)$ 전체가 $\max_a(\dots)$ 와 같아진다. 출처: 원서 그림 3-12.

그림 10의 논리를 식으로 옮기면 벨만 최적 방정식이다.

$$v_*(s) = \max_a \sum_{s'} p(s' | s, a) \{r(s, a, s') + \gamma v_*(s')\}. \quad (24)$$

상태 전이가 결정적이면 더 짧아진다.

$$v_*(s) = \max_a \{r(s, a, f(s, a)) + \gamma v_*(f(s, a))\}. \quad (25)$$

식 (11)과 식 (24)의 차이는 $\sum_a \pi(a | s)$ 자리에 $\max_a$ 가 들어간 것 하나뿐이다. 그러나 이 한 글자가 세 가지를 바꾼다.

1. **정책이 식에서 사라졌다.** 식 (24)에는 π 가 없다. 최적 정책이 무엇인지 모르는 채로 최적 가치를 구할 수 있고, 그 뒤에 정책을 뽑아내면 된다. 이것이 얻은 것이다.

2. **일차식이 아니다.** $\max$ 는 선형이 아니므로 §3.2처럼 연립일차방정식으로 묶어 한 번에 풀 수 없다. 이것이 잃은 것이다.
3. **반복 대입은 그대로 통한다.** 한 걸음마다 오차를 γ 배 이하로 줄이는 성질은 $\max$ 가 들어와도 그대로 남는다. 어느 행동이 이길지 미리 가정할 필요도 없다. $\max$ 가 매 반복 스스로 고르기 때문이며, §3.5.1에서 그 과정을 손으로 따라간다. 4.5절 가치 반복법이 이것이다.

3.4.2 행동 가치 함수의 벨만 최적 방정식

Q 에 대해서도 같은 치환이 성립한다. 식 (22)에서 다음 상태의 행동을 정책으로 평균내던 $\sum_{a'} \pi(a' | s')$ 를 $\max_{a'}$ 로 바꾸면 된다.

$$q_*(s, a) = \sum_{s'} p(s' | s, a) \left\{ r(s, a, s') + \gamma \max_{a'} q_*(s', a') \right\}. \quad (26)$$

식 (26)에서 $\max$ 가 안쪽에 있다는 점은 따로 기억해 둘 만하다. 6장 Q 러닝의 갱신식

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (27)$$

의 대괄호 안이 정확히 식 (26)의 우변에서 좌변을 뺀 것이다. 식 (22)의 SARSA 형태와 나란히 놓으면 두 알고리즘의 차이가 $\sum_{a'} \pi(a' | s')$ 대 $\max_{a'}$ 하나로 줄어든다.

3.5 벨만 최적 방정식의 예

3.5.1 벨만 최적 방정식 적용

같은 두 칸짜리 그리드 월드로 돌아간다. 이번에는 정책이 주어지지 않는다. 최적 정책이 무엇인지도 함께 찾아야 한다.

그림 3-13 두 칸짜리 그리드 월드

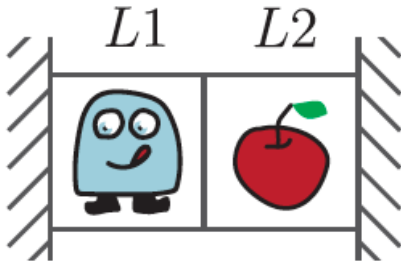


그림 3-14 상태 L1과 L2를 시작점으로 한 백업 다이어그램

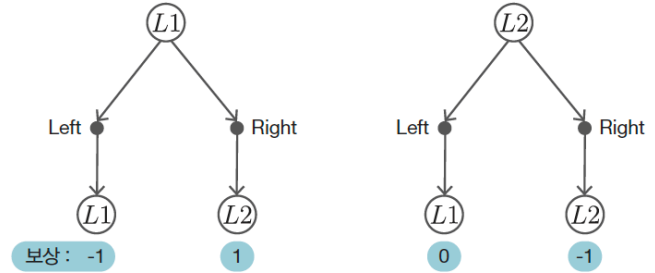


그림 11. 정책이 주어지지 않은 두 칸 세계(왼쪽)와 L1, L2에서 각각 시작하는 한 단계짜리 다이어그램(오른쪽). 그림 6 왼쪽에 있던 막대그래프가 사라졌고, 그림 7과 달리 가지에 확률 0.5가 붙어 있지 않다. 확률로 평균내는 대신 둘 중 큰 쪽을 고른다. 출처: 원서 그림 3-13, 3-14.

그림 11 오른쪽을 읽으면 두 식이 나온다.

$$\begin{cases} v_*(L1) = \max \{ -1 + 0.9 v_*(L1), 1 + 0.9 v_*(L2) \}, \\ v_*(L2) = \max \{ 0 + 0.9 v_*(L1), -1 + 0.9 v_*(L2) \}. \end{cases} \quad (28)$$

$\max$ 가 있는 채로는 풀 수 없으므로, 어느 쪽이 이길지 가정하고 풀어본 뒤 검증한다. L1에서 Right, L2에서 Left가 이긴다고 가정하면 $\max$ 가 사라져 일차식이 된다.

$$v_*(L1) = 1 + \gamma v_*(L2), \quad v_*(L2) = 0 + \gamma v_*(L1). \quad (29)$$

두 번째 식을 첫 번째에 대입하면

$$v_*(L1) = \frac{1}{1-\gamma^2} = 5.2632, \quad v_*(L2) = \frac{\gamma}{1-\gamma^2} = 4.7368 \quad (30)$$

이다. 이제 가정을 검증한다. 이 값을 식 (28)의 우변에 넣으면

$$\begin{aligned} L1 : \quad & -1 + 0.9 \times 5.2632 = 3.7368 < 1 + 0.9 \times 4.7368 = 5.2632 \quad \Rightarrow \text{Right } \checkmark \\ L2 : \quad & 0 + 0.9 \times 5.2632 = 4.7368 > -1 + 0.9 \times 4.7368 = 3.2632 \quad \Rightarrow \text{Left } \checkmark \end{aligned}$$

로 가정한 행동이 실제로 max를 달성한다. 따라서 식 (30)이 벨만 최적 방정식의 해다.

그림 3-16 두 칸짜리 그리드 월드의 최적 정책

그림 3-15 백업 다이어그램과 최적 상태 가치 함수

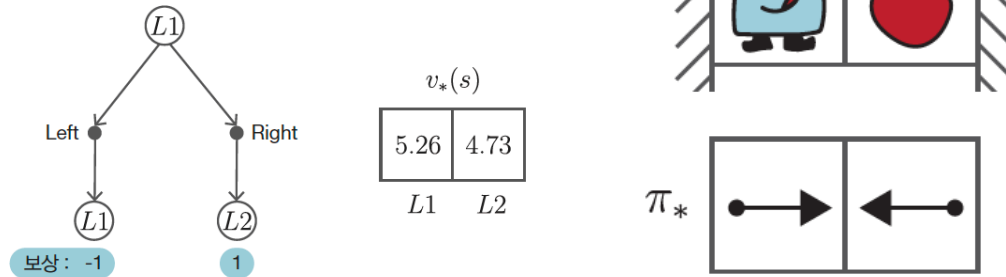


그림 12. 최적 상태 가치 함수(왼쪽)와 최적 정책(오른쪽). 앞 장에서 네 가지 정책을 전부 평가해 얻었던 값 및 정책 μ_2 와 같다. 이번에는 정책을 하나도 나열하지 않고 방정식만으로 얻었다. 출처: 원서 그림 3-15, 3-16.

손풀이가 통한 것은 상태가 두 개뿐이었기 때문이다. 가정할 것이 늘어나면 이 길은 곧 막힌다. 예제 s04_bellman_optimal.py는 그래서 가정을 세우지 않고 식 (25)의 우변에 그대로 반복 대입하여 같은 값에 도달한다. §3.2.2에서 식 (17)으로 했던 것과 같은 일이되, 행동마다 나온 값을 정책 확률로 평균내는 대신 그중 가장 큰 것을 고른다는 점만 다르다. 지금의 표로 새 표를 만들고 그것을 다시 넣는다는 열개는 그대로다.

이 반복은 손으로도 따라갈 수 있다. 상태 전이가 결정적이라 상태마다 후보가 둘뿐이므로, 두 후보를 계산해 큰 쪽을 다음 줄에 적는 것이 전부다. 표 8가 $v_0 = 0$ 에서 출발한 처음 여섯 번이다.

표 8. 두 칸 세계에서 식 (25)의 반복 대입을 손으로 돌린 것($\gamma = 0.9$). 가운데 두 묶음은 그 칸에서 계산되는 두 후보 $r + \gamma v_k(s')$ 이고, 굵은 값이 max로 뽑혀 다음 줄의 v_{k+1} 이 된다. 굵은 자리가 한 번도 바뀌지 않는 것과, 맨 아래줄에서 넣은 값과 뽑힌 값이 같아지는 것이 볼 곳이다.

k	현재 표 v_k		L1의 두 후보		L2의 두 후보	
	$v_k(L1)$	$v_k(L2)$	Left	Right	Left	Right
0	0.0000	0.0000	-1.0000	1.0000	0.0000	-1.0000
1	1.0000	0.0000	-0.1000	1.0000	0.9000	-1.0000
2	1.0000	0.9000	-0.1000	1.8100	0.9000	-0.1900
3	1.8100	0.9000	0.6290	1.8100	1.6290	-0.1900
4	1.8100	1.6290	0.6290	2.4661	1.6290	0.4661
5	2.4661	1.6290	1.2195	2.4661	2.2195	0.4661
∞	5.2632	4.7368	3.7368	5.2632	4.7368	3.2632

표 8에서 볼 것이 둘이다. 첫째, 굵은 자리가 $k = 0$ 부터 줄곧 (Right, Left)이며 한 번도 옮겨 다니지 않는다. 손풀이가 가정하고 사후에 검증해야 했던 바로 그 선택을 max가 매 반복 스스로 고르고 있는 것이다. 가정과 검증이라는 절차가 필요 없어지는 자리가 여기도. 둘째, 맨 아랫줄에서 넣은 표 (5.2632, 4.7368)과 뽑혀 나온 표가 정확히 같다. 넣은 것이 그대로 나왔으므로 이 표가 §3.2.2에서 말한 고정점이고, 그것이 곧 v_* 다.

예제는 갱신량이 정확히 0이 될 때까지 돌아 333회에서 멈춘다. 알고리즘이 수렴을 판정한 것이 아니라, 갱신량 γ^k 가 배정밀도로 표현할 수 있는 간격 아래로 내려가 더 이상 적을 다음 값이 없어 멈춘 것이다. 실제로는 이렇게 0을 기다리기보다 문턱 ε 을 정해 두고 갱신량이 그 아래로 내려가면 멈추는 편이 낫다.

같은 반복을 3×4 그리드에 적용하면 6회 만에 끝난다. 두 칸 세계와 달리 목표 상태라는 흡수 상태가 있어 가치가 유한 걸음 안에 확정되기 때문이며, 반복 횟수를 지배하는 것이 γ 만은 아님을 보여준다.

[보충] 나머지 세 가정을 택했다면

앞의 손풀이는 “L1에서 Right, L2에서 Left”라는 가정에서 출발했다. 이 가정이 맞았음을 사후 검증했지만, 다른 세 가정을 택했다면 어떤 일이 벌어졌을지는 확인하지 않았다. 결정적 정책이 네 가지뿐이므로 전부 풀어 볼 수 있다. 가정한 정책 μ 에 대해 max를 제거한 연립 일차방정식 $(I - \gamma P_\mu)v = r_\mu$ 를 풀고, 그 해가 식 (28)의 max를 실제로 달성하는지 검사한 결과가 표 9다.

표 9. 네 가지 결정적 정책 가정에 대한 해와 자기일관성 검사($\gamma = 0.9$). “달성 행동” 열은 그 해를 식 (28)의 우변에 넣었을 때 실제로 max를 주는 행동이다. 가정과 일치하는 것은 한 행뿐이다.

가정 ($\mu(L1), \mu(L2)$)	$v(L1)$	$v(L2)$	달성 행동	자기일관
(Left, Left)	-10.0000	-9.0000	(Right, Left)	모순
(Left, Right)	-10.0000	-10.0000	(Right, Left)	모순
(Right, Left)	5.2632	4.7368	(Right, Left)	일관
(Right, Right)	-8.0000	-10.0000	(Right, Left)	모순

표 9는 손풀이의 논리 구조를 드러낸다. 벨만 최적 방정식을 손으로 푼다는 것은 $|A|^{|\mathcal{S}|}$ 개의 경우 각각에 대해 선형 방정식을 풀고 자기일관성을 검사하는 일과 같으며, 그중 정확히 하나가 통과한다. 경우의 수는 전수 탐색과 같지만 각 경우의 비용이 무한 궤적 전개에서 선형 방정식 풀이로 줄었다는 것이 벨만 방정식이 준 것이다. 그리고 반복 대입은 이 경우 분석을 아예 건너뛴다. max가 매 반복마다 스스로 옳은 가치를 골라 주기 때문이다.

세 모순 행의 값들도 읽어 둘 만하다. -10은 $-1/(1 - \gamma)$ 로, 영원히 벽에 부딪히며 -1을 받는 정책의 가치다. (Left, Right)는 두 상태 모두 각자의 벽에 갇히는 정책이라 양쪽 다 -10이 된다. 가정이 나쁘면 방정식은 여전히 답을 주되 그 답은 “그 나쁜 정책의 가치”일 뿐이며, 자기일관성 검사만이 그것을 걸러낸다. 검사를 생략하면 max를 임의로 제거한 식의 해를 최적 가치로 착각하게 된다.

3.5.2 최적 정책 구하기

최적 가치를 알면 최적 정책은 탐욕적으로 뽑는다. Q 가 있으면 한 줄이다.

$$\mu_*(s) = \arg \max_a q_*(s, a). \quad (31)$$

V 만 있으면 한 걸음 내다봐야 한다.

$$\mu_*(s) = \arg \max_a \sum_{s'} p(s' | s, a) \{ r(s, a, s') + \gamma v_*(s') \}. \quad (32)$$

식 (32)의 우변에 p 와 r 가 등장한다는 것이 표 4가 말한 Q 의 장점이 드러나는 자리다. 환경 모델을 모르면 V 에서 정책을 뽑을 수 없다.

두 칸 세계에서 식 (32)를 적용하면 L1에서 Right, L2에서 Left가 나온다(그림 12 오른쪽). 앞 장이 네 가지 정책을 전부 평가해 골라냈던 μ_2 와 같은 정책이며, 사과를 먹고 되돌아오기를 반복하는 행동이다.

세 방법의 대조

예제 s04_bellman_optimal.py는 같은 답에 이르는 세 가지 경로를 나란히 놓고 assert로 일치를 확인한다. 표 10가 그 대조다.

표 10. 두 칸 세계의 최적 가치를 구하는 세 방법($\gamma = 0.9$). 세 값이 모두 일치한다.

방법	$v_*(L1)$	$v_*(L2)$	비용
반복 대입 (식 (25))	5.2632	4.7368	333회 $\times$ $ S A $
손풀이 (식 (30))	5.2632	4.7368	가정 + 검증
전수 탐색 (앞 장)	5.2632	4.7368	$ A ^{ S }$ 개 정책 $\times$ 500걸음

값이 같다는 사실보다 중요한 것은 비용 열이다. 전수 탐색은 정책 수가 $|A|^{|S|}$ 로 폭발하고 각 정책마다 궤적을 500걸음 펼쳐야 한다. 벨만 최적 방정식의 반복 대입은 상태 수에 비례하는 계산을 정해진 횟수만큼 되풀이할 뿐이다. 상태가 두 개일 때는 둘 다 순식간이지만, 상태가 100개가 되면 한쪽은 4^{100} 이고 다른 쪽은 $100 \times 4 \times K$ 다.

3.6 정리

본고는 벨만 방정식이 앞 장에서 만난 두 벽을 어떻게 허무는지를 정리하였다.

출발점은 수익의 재귀 구조 $G_t = R_t + \gamma G_{t+1}$ 한 줄이었다(식 (6)). 이를 가치 함수의 정의에 대입하고 정책과 상태 전이라는 두 층의 확률로 전개하면 벨만 방정식 (식 (11))을 얻는다. 이 식은 무한한 미래를 “한 걸음 보상 + $\gamma \times$ 이웃 상태의 가치”로 접어 넣으므로, 가치 함수는 미지수 $|S|$ 개짜리 연립일차방정식의 해가 된다. 상태마다 식을 한 줄씩 쓰면 되고, 미지수의 개수가 상태의 개수를 넘지 않으므로 언제나 유한하다. 앞 장이 500걸음 시뮬레이션으로 얻었던 5.2632가 2×2 연립방정식 한 번으로 나온 것이 그 요약이다.

행동 가치 함수 $q_\pi(s, a)$ 는 첫 행동만 정책에서 떼어낸 것으로, V 와는 식 (20), (21)으로 자유롭게 오간다. $|A|$ 배의 기억 용량을 쓰는 대신 환경 모델 없이 $\arg \max_a q(s, a)$ 로 행동을 고를 수 있다는 것이 Q 의 값어치이며, 5장 이후 모델 없는 기법들이 예외 없이 Q 를 추정하는 근거다. 벨만 최적 방정식은 $\sum_a \pi(a | s)$ 자리를 $\max_a$ 로 바꾼 것으로 (식 (24), (26)), 정책이 식에서 사라지는 대신 선형성을 잃는다. 최적 가치를 알고 나면 최적 정책은 $\mu_*(s) = \arg \max_a q_*(s, a)$ 로 탐욕적으로 뽑으면 되고, 두 칸 세계에서 그 답은 $v_*(L1) = 5.2632$, $v_*(L2) = 4.7368$, $\pi_* = (\text{Right}, \text{Left})$ 다. 앞 장이 정책 네 가지를 전부 평가해 얻은 것을 이 장은 방정식 두 줄로 얻었다.

본문 안에 “[보충]”으로 남긴 논의 가운데 둘은 곧장 다음 장으로 이어진다.

하나는 무작위 정책의 Q 를 한 번 탐욕화해 본 것이다 (§3.3.2). 3×4 그리드의 10개 상태 전부에서 가치가 올랐고 평균은 -0.212 에서 0.760 으로 바뀌었다. 최댓값은 평균보다 작을 수 없다는 사실 하나에서 나오는 정책 개선 정리의 내용이다. 그러나 아래쪽 세 칸은 최적값에 최대 0.279 못 미치는데, 탐욕화의 근거가 된 Q 가 여전히 낡은 정책의 것이기 때문이다. 이 간극이 4장 정책 반복법이 왜 “한 번”이 아니라 “반복”인지를 말해 준다.

다른 하나는 손풀이가 전제한 네 가지 가정을 전부 풀어 본 것이다 (§3.5.1). 자기일관인 것은 (Right, Left) 하나뿐이었다. 나머지 셋도 방정식은 답을 주지만 그것은 그 나쁜 정책의 가치일 뿐이며, 자기일관성 검사만이 그것을 걸러낸다. 벨만 최적 방정식을 손으로 푼다는 것은 경우를 나열해 검사하는 일인 셈인데, 반복 대입은 이 경우 분석을 통째로 건너뛴다. $\max$ 가 매 반복 스스로 옳은 가치를 고르기 때문이다(표 8).

다음 장에서는 이 방정식들을 반복 계산으로 푸는 동적 프로그래밍을 다룬다. 벨만 방정식의 반복이 정책 평가이고 벨만 최적 방정식의 반복이 가치 반복법이며, 표 6에 남은 0.0668을 회수하기 위해 평가와 탐욕화를 번갈아 돌리는 것이 정책 반복법이다.

재현 정보

본고의 모든 수치는 Python 3.12.13, NumPy 2.5.2, Matplotlib 3.11.1 환경에서 재현하였다. 예제는 난수를 쓰지 않으므로 시드 고정 없이 항상 같은 값이 나온다.

표 11. 예제 코드 목록. 원서 3장에는 소스 코드가 없으므로 모두 본고의 보충 예제다.

파일	내용	대응 절
ch03/s01_bellman_linear.py	벨만 방정식을 연립해 푼 정확해	§3.2.1
ch03/s02_bellman_iterative.py	반복 대입과 오차의 지수적 감소	§3.2.2
ch03/s03_q_function.py	$V \leftrightarrow Q$ 변환과 식 (22) 검증	§3.3
ch03/s04_bellman_optimal.py	벨만 최적 방정식의 반복 대입, 손풀이, 전수 탐색 대조	§3.5

표 1, 2, 5, 10과 그림 8은 표 11의 예제를 그대로 실행해 얻은 출력이다. 표 8, 6, 7, 9는 본고를 위해 별도로 계산한 값이다.

표 6의 $v_{\pi'}$ 는 탐욕화한 정책의 벨만 방정식을 연립해 정확히 푼 값이며, v_* 는 같은 환경에 식 (25)의 반복 대입을 수렴할 때까지 (6회) 적용해 얻었다. 표 9는 네 가지 결정적 정책 각각에 대해 $(I - \gamma P_{\mu})v = r_{\mu}$ 를 풀고 그 해를 식 (28)의 우변에 대입하여 $\arg \max$ 를 확인한 결과다.

3×4 그리드의 상태는 원서와 예제 코드의 표기를 따라 (h, w) 로 적었다. h 가 위에서부터 센 행, w 가 왼쪽에서부터 센 열이며 (0, 3)이 목표, (1, 1)이 벽, (1, 3)이 폭탄이다.

참고문헌

- [1] 사이토 고키, 밑바닥부터 시작하는 딥러닝 4: 파이썬으로 구현하는 강화 학습 알고리즘, 한빛 미디어, 2023.
- [2] sajacaros, 마르코프 결정 과정과 최적 정책: 상태가 개입할 때 무엇이 달라지는가, 2026.
- [3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.
- [4] R. Bellman, *Dynamic Programming*, Princeton University Press, 1957.
- [5] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*, Wiley, 1994.

본 문서에 실린 그림 중 “원서 그림”으로 표기한 것은 참고문헌 [1]의 도판이며, 학습 정리 목적으로 인용하였다. 그 밖의 그림과 표는 본고에서 직접 생성한 것이다.